

Reproducing LoRA: Cross-Dataset Rank-Sensitivity Study

Mutasem Usama Taha — LebNet Tech Fellows 2026

August 2026

1. Project Title & Abstract

Reproducing LoRA: Cross-Dataset Rank-Sensitivity Study *LebNet Tech Fellows 2026 — Option 1: Reproduce an Academic Paper + Extend It*

This project reproduces the core fine-tuning methodology of **LoRA: Low-Rank Adaptation of Large Language Models** (Hu et al., *ICLR 2022*) on RoBERTa-base, and extends it with an original **cross-dataset rank-sensitivity study**. LoRA freezes a pretrained model’s weights and injects small trainable low-rank matrices into selected attention projections, dramatically reducing the number of trainable parameters needed for downstream fine-tuning. We fine-tune RoBERTa-base with LoRA adapters on the query and value attention projections across five ranks ($r \in \{1, 2, 4, 8, 16\}$) and three seeds per rank, on two classification tasks of different complexity — SST-2 (binary sentiment) and AG News (four-class topic classification) — compared against a full-fine-tuning baseline for each dataset. This produced a 30-run LoRA experimental matrix plus two full-FT baselines. The central finding is that LoRA is highly parameter-efficient on both datasets (under 1% of full fine-tuning’s trainable parameters at every rank tested) and that increasing rank from 1 to 16 produces only marginal accuracy gains. The project’s original hypothesis — that the more complex four-class AG News task would require substantially higher LoRA rank than binary SST-2 — is **not supported** by the collected data.

2. Introduction & Problem Statement

Fine-tuning large pretrained language models traditionally updates every parameter of the model. This is tractable for a $\sim 125\text{M}$ -parameter model like RoBERTa-base, but the same approach becomes increasingly expensive as model scale grows and requires storing a full, separately fine-tuned checkpoint for every downstream task. **Low-Rank Adaptation (LoRA)** (Hu et al., 2022) addresses this by freezing the pretrained weights and injecting small trainable low-rank update matrices into selected layers, so that only a tiny fraction of parameters need to be trained and stored per task.

The rank r of these update matrices is LoRA’s central hyperparameter: a smaller r means fewer trainable parameters, but potentially less capacity to adapt the model. This project asks a concrete, practical question about that trade-off:

Research question: How sensitive is downstream classification performance to LoRA rank, and does task complexity affect the rank required for strong performance?

The reproduction target is SST-2, the setting closest to the original paper’s GLUE experiments. **SST-2** was selected because it lets the project directly reproduce LoRA’s core fine-tuning mechanism on a benchmark central to the original paper. The project then extends the evaluation to **AG News**, a four-class topic-classification task, chosen specifically because its higher class count and different domain (news topics vs. sentiment) provide a natural test of whether a more complex classification task demands a higher LoRA rank — this comparison is the project’s original

extension beyond the paper. The guiding hypothesis, stated in the original proposal and tested empirically rather than assumed, was:

“A low rank ($r \leq 4$) should be sufficient for binary classification such as SST-2, while the more complex four-class AG News task may require a higher rank to match full fine-tuning performance.”

3. Methodology (Approach / Reproduction Details)

Backbone: RoBERTa-base (~125M parameters), used both as the frozen backbone for LoRA runs and as the fully trainable model for full-fine-tuning baselines.

LoRA mechanism reproduced: following Hu et al. (2022), a pretrained weight matrix W is left frozen and a low-rank update $\Delta W = B \cdot A$ is learned and added alongside it, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and rank r is much smaller than $\min(d, k)$. LoRA adapters were applied only to the **query and value attention projections** of each self-attention layer — the target modules used in the original paper’s own ablations — with `lora_alpha = 16` and adapter dropout 0.1.

Experimental matrix: 5 ranks $\times$ 3 seeds $\times$ 2 datasets = **30 LoRA runs total** (15 SST-2 + 15 AG News), plus one full-fine-tuning baseline per dataset (seed 42). LoRA runs used learning rate $2e-4$; full-fine-tuning baselines used $2e-5$. All runs shared batch size 32 (eval batch size 64), 3 epochs, and max sequence length 128. Each seed re-initializes Python, NumPy, and PyTorch CPU/CUDA RNGs before model or data construction, so the three runs per rank/dataset are fully independent.

4. Implementation Details & Results

The implementation is a standard Hugging Face transformers + peft training pipeline (`src/train.py`, `src/model.py`, `src/data.py`), invoked entirely through CLI arguments, with every run’s full effective configuration, parameter counts, timing, and evaluation metrics written to a per-run JSON in `results/raw/`. Figures and summary tables are generated from those JSONs by `experiments/generate_report_figures.py`.

Main results (mean validation accuracy; LoRA is mean across 3 seeds, full-FT is a single seed):

Dataset	r=1	r=2	r=4	r=8	r=16	Full FT
SST-2	0.9354	0.9346	0.9362	0.9365	0.9373	0.9438
AG News	0.9408	0.9421	0.9428	0.9426	0.9433	0.9542

Rank sensitivity: $r=16$ achieved the best mean accuracy on both datasets, but the gain over $r=1$ was small — **+0.19 percentage points on SST-2** and **+0.25 percentage points on AG News**. On SST-2 the relationship was not even monotonic ($r=2$ dipped slightly below $r=1$); on AG News it rose from $r=1$ through $r=4$, dipped slightly at $r=8$, then reached its maximum at $r=16$. In both cases, the magnitude of the rank effect was comparable to, or smaller than, seed-to-seed standard deviation at a single rank.

Parameter efficiency: LoRA used **less than 1% of full fine-tuning’s trainable parameters at every rank tested** — 630,532 params (0.50%) at $r=1$ rising to 1,183,492 params (0.94%) at $r=16$, against 124,648,708 for full fine-tuning — while landing within roughly 0.7-1.4 accuracy points of the full-fine-tuning baseline on both datasets.

Experimental issue and correction — AG News full fine-tuning learning rate: an initial AG News full-fine-tuning attempt used learning rate $2e-4$ (the LoRA learning rate) instead of a rate suited to updating all 124.6M parameters. The run collapsed to predicting a single constant class (accuracy exactly at the 0.2500 random-guess level for the balanced four-class split, with training loss flat near $\ln(4) \approx 1.386$ for all three epochs). This failed run was preserved on disk as documented evidence and is **not** used as a baseline anywhere in the results above. The root cause was that `configs/*.yaml` files document intended hyperparameters but are not actually loaded by

src/train.py — every run’s parameters come from CLI arguments, and the correct $2e-5$ rate, though already documented in configs/full_finetuning.yaml, was not the rate actually passed to that run. The corrected run, using $lr=2e-5$, trained normally and is the AG News full-fine-tuning baseline used throughout this report.

Reproducibility note — timing anomalies: several training runs on both datasets showed single-epoch wall-clock spikes, the largest being roughly 8 hours 17 minutes in one AG News LoRA run ($r=16$, seed=123), against 15–18 minutes for that run’s other epochs. Every anomaly was checked against checkpoint file modification timestamps, which matched the recorded gap, confirming a genuine wall-clock event — consistent with the host machine sleeping or suspending — rather than a bug. All affected runs completed with valid metrics and correct parameter counts. This is reported purely as a reproducibility note about the recorded timing fields; it is **not** evidence about the computational complexity of any rank or dataset, and no timing-vs-rank comparison is drawn from it.

5. Discussion & Analysis

Comparing the two datasets surfaces two findings that should not be conflated. First, AG News reaches a higher absolute accuracy than SST-2 at every rank, and its gap to its own full-fine-tuning ceiling (1.1–1.4 points) is larger than SST-2’s (0.65–0.92 points) — this reflects the two tasks’ different intrinsic difficulty and ceilings, not a statement about rank sensitivity. Second, and central to the hypothesis under test, AG News does **not** show stronger rank sensitivity than SST-2: its rank-to-rank accuracy spread (0.0025) is comparable to SST-2’s (0.0027), and its per-rank seed variance is actually tighter (std range 0.0003–0.0009 vs. SST-2’s 0.0024–0.0053).

Hypothesis outcome: the results do **not support** the hypothesis that the more complex four-class AG News task requires substantially higher LoRA rank than binary SST-2. $r=1$ already captures most of the achievable performance on both datasets (within 0.19–0.25 points of each dataset’s own $r=16$ result), and if AG News genuinely needed higher rank, its $r=1 \rightarrow r=16$ improvement would be expected to be markedly larger than SST-2’s — instead it is only modestly larger (0.25 vs. 0.19 points), and both are small. This is reported as a cautious, descriptive finding from a three-seed experimental matrix, not as a definitive disproof: the sample size is sufficient to see that the rank effect is small relative to seed-to-seed noise on this data, but not large enough to rule out a small real effect the experiment was underpowered to detect.

Limitations: only three seeds per rank/dataset for the LoRA sweep; only one full-fine-tuning seed per dataset (no seed-based error bar on the full-FT reference); only two datasets, both English-language text classification tasks differing mainly in class count; only one backbone model (RoBERTa-base); rank range limited to $\{1, 2, 4, 8, 16\}$; several runs’ wall-clock timing fields were affected by host-machine sleep/suspend events and were excluded from timing analysis; and no formal statistical significance testing (t-tests, confidence intervals) was performed beyond descriptive mean/std comparisons.

6. Reflection on Learnings

The most rewarding part of the project was watching a fairly large, controlled experimental matrix — 30 LoRA runs across two datasets, five ranks, and three seeds — converge into a clear, internally consistent answer to a question that was genuinely open going in. The proposal explicitly framed the higher-rank-for-AG-News idea as a hypothesis to test, not an assumption, and it was satisfying to have the completed data actually refute it with a coherent, explainable pattern rather than a noisy or ambiguous one.

The most challenging technical problem was the AG News full-fine-tuning collapse: a run that silently converged to predicting a single constant class because it was launched with the LoRA learning rate ($2e-4$) instead of a full-fine-tuning-appropriate rate ($2e-5$). Diagnosing it meant recognizing that a flat loss near $\ln(4)$ and an accuracy of exactly 0.25 on a balanced four-class task were not coincidental, but the fingerprint of a degenerate, constant-output classifier. Tracing

the root cause further — to the fact that `configs/*.yaml` were documentation-only and never actually loaded by `train.py` — was the more valuable lesson: a config file isn't trustworthy unless the code enforces it, even when the "correct" value was written down all along. The fix was procedural as much as numerical: preserving the failed run as evidence instead of deleting it, and explicitly excluding it from every baseline and figure, so the mistake stayed visible rather than quietly erased.

A second, more unusual difficulty was a set of large single-epoch timing spikes, including one roughly 8-hour-17-minute gap. Rather than assuming a bug or discarding the run, the anomaly was checked independently against checkpoint file modification timestamps, confirming a genuine wall-clock gap (consistent with the host machine sleeping), not a defect in the training loop. Excluding the affected timing fields from any computational-cost analysis, rather than letting them contaminate a plot, was its own small lesson in being precise about what a piece of evidence can and cannot support.

Together, these experiences reinforced a few broader points about reproducibility and ML experimentation: documented configuration is only trustworthy if the code actually enforces it; unexpected results are usually worth diagnosing rather than dismissing, since they often reveal a concrete, fixable cause; preserving failed or anomalous runs as evidence, instead of silently rerunning and discarding them, keeps a project's record honest and auditable; and a hypothesis stated up front and genuinely tested against multi-seed data — rather than a single run — can produce a real, sometimes counter-intuitive finding, which is more valuable than confirming an assumption would have been.